

# **JavaScript Core Concepts Document**

## **1. Introduction**

- Overview of JavaScript Execution Environments
- JavaScript in React and Browser

## **2. Adding JavaScript to HTML**

- Inline JavaScript vs. External JavaScript Files
- Script Tag Attributes (`defer`, `type="module"`)

## **3. React Projects & Build Process**

- Why React Code Works Without Explicit `<script>` Tags
- How the Build Process Works
- Code Optimization for Production
- Tools Enabling the Build Process

## **4. JavaScript Import & Export Syntax**

- Named Exports & Imports
- Default Exports & Imports
- Mixing Named & Default Exports
- Importing Everything as an Object (`* as`)
- Aliasing Imports (`as` Keyword)

## **5. JavaScript Fundamentals**

- Data Types in JavaScript
- Declaring Variables (`let`, `const`)
- Best Practices for Variable Usage

## **6. JavaScript Operators**

- Mathematical Operators
- String Concatenation
- Comparison Operators (`===`, `>`, `<`, `>=`, `<=`)
- Conditional Execution (`if`, `else if`, `else`)

## **7. JavaScript Functions**

- Function Definition & Execution
- Function Parameters & Arguments
- Default Parameters
- Returning Values from Functions
- Arrow Functions and Implicit Returns

## **8. JavaScript Objects and Classes**

- Object Structure and Dot Notation
- Methods in Objects
- Using `this` Keyword
- Classes as Blueprints for Objects

## **9. JavaScript Arrays & Utility Methods**

- Array Basics & Accessing Elements
- Common Methods (`push()`, `findIndex()`, `map()`)
- Transforming Arrays with `map()`

## **10. Destructuring in JavaScript**

- Destructuring Arrays
- Destructuring Objects
- Using Aliases in Object Destructuring

## **11. Spread Operator in JavaScript**

- Merging Arrays with Spread Operator
- Merging Objects with Spread Operator

## **12. Control Structures in JavaScript**

- `if` Statements & Conditional Execution
- Looping Through Arrays (`for...of`)

## **13. JavaScript vs. React for DOM Manipulation**

- Selecting & Modifying Elements in JavaScript

- React's Declarative Approach to UI Updates

#### **14. Passing Functions as Values**

- Functions as First-Class Citizens
- Using `setTimeout()` with Function Arguments
- Passing Functions to Custom Functions

#### **15. Functions Inside Functions (Nested Functions)**

- Scope & Execution of Inner Functions
- Usage in React and Modular Code

#### **16. Reference vs. Primitive Values**

- Difference Between Primitive and Reference Values
- Why Objects & Arrays are Mutable
- `const` and Object Mutation

## JavaScript Core Concepts

### JavaScript Execution Environments

- JavaScript can be executed in multiple environments:
  - **Browser** – Original execution environment.
  - **Node.js & Deno** – Allows JavaScript execution outside the browser.
  - **Mobile Apps** – Possible with **Capacitor** and **React Native**.

### JavaScript in React

- **React** is a **front-end, client-side library** that builds user interfaces.
- React can be used with **React Native** to build native mobile apps.

### JavaScript in the Browser

- The **general syntax and rules** remain the same regardless of the environment.
- JavaScript code can be **added to a website** using the **<script>** tag.

### Adding JavaScript to HTML

#### 1. Inline JavaScript

```
<script>  
  console.log("Hello, world!");  
</script>
```

- Leads to **unmaintainable** HTML files.
- Only used for **short scripts**.

## 2. External JavaScript File

- JavaScript is stored in a **.js file** (e.g., `app.js`).
- Linked to HTML using the **<script>** tag:

```
<script  
  src="assets/scripts/app.js"></script>
```

- **More maintainable** for large projects.

## Script Tag Attributes

### 1. **defer** Attribute

- Ensures JavaScript **executes only after** the HTML document is fully parsed.
- Prevents issues when accessing **DOM elements** before they are loaded.
- Example:

```
<script src="app.js" defer></script>
```

### 2. **type="module"** Attribute

- Treats JavaScript file as a **module**, enabling **import/export** syntax.
- Example:

```
<script type="module" src="app.js"></script>
```

- Allows importing functions from another script:

```
// util.js  
export function greet() {  
  console.log("Hello from module!");  
}  
  
// app.js  
import { greet } from './util.js';  
greet();
```

## **React Projects & Build Process**

- In **React projects**, script tags **are not manually added** to HTML.
- React uses a **build process** that **automatically injects script tags** into HTML.
- The next topic will cover **why React projects use a build process**.

## React Build Process & Execution in the Browser

### Why React Code Works Without Explicit `<script>` Tags

- React applications execute **JavaScript code**, but they do not explicitly include `<script>` tags in the source code.
- React projects use a **build process** that transforms the written code before execution in the browser.
- The **final output** injected into the browser differs from the original code due to this transformation.

### How the Build Process Works

- A background tool manages the build process.
- Dependencies and tools are listed in **`package.json`**, including:
  - **React Libraries** (`react`, `react-dom`)
  - **React Scripts** (`react-scripts`), which:
    - **Transforms** React code before execution.
    - **Injects** the processed code into the browser.

### How Script Tags Appear in the Browser

- The **development server** automatically runs in the background.
- The transformed code is **dynamically injected** into the HTML file via `<script>` tags.
- The **browser's Developer Tools** reveal these script tags, which were **generated** by the build process.

### Reasons for Using a Build Process

1. **React Code is Not Natively Executable**
  - React uses **JSX (JavaScript XML)**, an **HTML-like syntax** in JavaScript files.

- Browsers **do not support JSX** natively.
- JSX must be **converted** into standard JavaScript before execution.

### Example: JSX vs. Transformed JavaScript

```
// JSX (Not supported by browsers)
const element = <h1>Hello, React!</h1>;
// Transformed JavaScript (Valid in browsers)
const element = React.createElement("h1", null,
"Hello, React!");
```

## 2. Code Optimization for Production

- **Minification** shortens variable and function names to reduce file size.
- **Smaller file sizes = Faster website loading speeds.**

### Example: Unoptimized vs. Minified Code

```
function sayHello() {
  console.log("Hello, React!");
}
function a(){console.log("Hello, React!")}
```

## Tools That Enable the Build Process

- **Create React App (CRA), Vite, or CodeSandbox** include an automatic build process.
- **Node.js is required** for React projects because:
  - It enables **package management** with npm or npx.
  - It runs tools like **Babel (JSX to JavaScript compiler)** and **Webpack (bundler)**.

## Key Takeaways



- React code requires **transformation** before execution in the browser.
- The build process **automates JSX conversion** and **optimizes code**.
- **Development servers watch** for changes and update transformed code dynamically.
- **Node.js is essential** for React development to enable the build process.

## JavaScript Import & Export Syntax

In advanced JavaScript projects like **React apps**, it is best practice to **split code across multiple files** for maintainability. This is achieved using **import** and **export** keywords.

---

### Named Exports & Imports

- Used when exporting **multiple variables/functions** from a file.
- Add `export` before the variable or function.

#### Example (Named Export)

```
// util.js
export const apiKey = "123456ABC";
export function fetchData() {
  console.log("Fetching data...");
}
```

#### Example (Named Import)

```
// app.js
import { apiKey, fetchData } from "./util.js";

console.log(apiKey); // Output: 123456ABC
fetchData(); // Output: Fetching data...
```

- ✓ **Curly braces { }** are required in named imports.
- ✓ **JavaScript is case-sensitive** (import names must match the export names exactly).
- ✓ **Relative paths** are used:
  - `./` → Same directory

- `../` → Move up a directory
- 

## Default Exports & Imports

- Used when exporting **only one main function/variable** per file.
- `export default` is used **without a name**.
- Only **one default export** is allowed per file.

### Example (Default Export)

```
// util.js
export default "123456ABC";
```

### Example (Default Import)

```
// app.js
import key from "./util.js";

console.log(key); // Output: 123456ABC
```

- ✓ **Curly braces { }** are **NOT used** in default imports.
  - ✓ Any name can be assigned to the imported value.
- 

## Mixing Named & Default Exports

A file can **have both** a default export and multiple named exports.

### Example (Mixed Export)

```
// util.js
```

```
export default "123456ABC"; // Default export
export const apiKey = "DEF789"; // Named export
export function fetchData() {
  console.log("Fetching data...");
}
```

### Example (Mixed Import)

```
// app.js
import key, { apiKey, fetchData } from "./util.js";

console.log(key); // Output: 123456ABC
console.log(apiKey); // Output: DEF789
fetchData(); // Output: Fetching data...
```

- ✓ Default import **must come first**, followed by named imports.
- 

### Importing Everything as an Object (\* as)

Instead of listing all named exports, we can **group them into an object**.

### Example (Import All)

```
// app.js
import * as util from "./util.js";

console.log(util.apiKey); // Output: DEF789
util.fetchData(); // Output: Fetching data...
```

- ✓ Default export is available as `util.default`.
- ✓ Named exports are accessed using **dot notation** (`util.apiKey`).

---

## Aliasing Imports (as Keyword)

- Used to rename imports for clarity or avoiding name conflicts.

### Example (Aliasing Named Imports)

```
// app.js
import { apiKey as key, fetchData as fetch } from
"./util.js";

console.log(key); // Output: DEF789
fetch(); // Output: Fetching data...
```

- ✓ **Original names remain unchanged in the export file.**
- ✓ **Useful when multiple files export similar variable names.**

---

## Important Notes

1. **type="module" is required** when using import/export in vanilla JavaScript:
2. `<script type="module" src="app.js"></script>`
  - React projects **don't require** this because the build process **bundles files**.
3. **Browser Compatibility:**
  - Older browsers **don't support** import/export syntax.
  - React's build process converts imports/exports into a format **compatible with all browsers**.

---

## Key Takeaways

- ✓ Use named exports (`export { }`) for multiple values per file.
- ✓ Use default exports (`export default`) when exporting a single main value.
- ✓ Import named exports using `{ }` and default exports without `{ }`.
- ✓ Use `*` `as` to import everything as an object.
- ✓ Use `as` to rename imports for better readability.
- ✓ Use `type="module"` in vanilla JavaScript but not in React.

 This syntax is widely used in JavaScript and React projects, making it essential to master!

# JavaScript Fundamentals: Variables, Values & Operators

## Data & Values in JavaScript

- JavaScript applications are **data-driven** (e.g., tweets, locations).
- Different **data types**:
  - **Strings** → "Hello, World!"
  - **Numbers** → 42, 3.14
  - **Booleans** → true, false
  - **Null** → Explicit absence of a value (`let x = null;`)
  - **Undefined** → Variable declared but not assigned a value (`let x;`)
  - **Objects** → Collection of key-value pairs.

## Creating & Using Values

- Values can be created **directly**:
- `console.log("Hello, World!");`
- **Variables** store values for reuse and better readability.

## Declaring Variables

- **let** → Used for variables that may change.
- **const** → Used for constants that should not be reassigned.
- **Naming rules**:
  - Use **camelCase** (`userMessage`).
  - No special characters except `_` and `$`.
  - Cannot start with a number.
  - No whitespace or dashes.

Example:

```
let userMessage = "Hello, World!";
```

```
console.log(userMessage);  
console.log(userMessage);
```

- **Advantage:** Changing the value in one place updates all references.

## Constants (const)

- **Immutable** once assigned.
- Prevents accidental reassignment.

Example:

```
const greeting = "Hello, World!";  
console.log(greeting);  
greeting = "New Message"; // ❌ Error: Assignment to  
constant variable
```

- Use **const** when a value should not change.
- Use **let** if reassignment is needed.

## Best Practices

- **Prefer const** for immutable values.
- Use **let** only when reassignment is necessary.
- Improves **code clarity** by signaling intent.



## JavaScript Operators

### Mathematical Operators

- Perform basic arithmetic operations:
  - **Addition (+)** → 10 + 5 // 15
  - **Subtraction (-)** → 10 - 5 // 5
  - **Multiplication (\*)** → 10 \* 5 // 50
  - **Division (/)** → 10 / 5 // 2

### String Concatenation (+)

- The **+** operator is also used for combining (**concatenating**) strings:

```
console.log("Hello" + "World"); // Output:
HelloWorld
console.log("Hello" + " World"); // Output: Hello
World
```

- **Numbers vs. Strings:**
  - Strings are enclosed in **quotes** ("text" or 'text').
  - Numbers are **not** enclosed in quotes (10, 5.5).

### Comparison Operators

- **Triple Equals (===)** → Strict equality check (value **and** type must match).

```
console.log(10 === 10); // true
console.log(10 === "10"); // false (number vs.
string)
```

- **Greater/Less Than Operators:**
  - **>** → Greater than
  - **<** → Less than

- `>=` → Greater than or equal to
- `<=` → Less than or equal to

```
console.log(10 > 5); // true
console.log(10 < 5); // false
console.log(10 >= 10); // true
console.log(10 <= 5); // false
```

## **Conditional Execution (if statements)**

- `if` statements execute code **only if a condition is met**.

```
if (10 === 10) {
  console.log("Condition is true!"); //
Output: Condition is true!
}
```

- Conditions are **usually dynamic** (e.g., user input) instead of hardcoded values.
- In **React apps**, conditions help control UI rendering.

# JavaScript Functions

## Function Definition & Execution

- A **function** is a block of reusable code that executes when invoked.
- Functions are defined using the `function` keyword or **arrow functions** (`=>`).
- Code inside a function is not executed immediately but only when **called**.

### **Example:**

```
function greet() {  
    console.log("Hello!");  
}
```

```
greet(); // Calling the function → Output: Hello!
```

- Functions can be called multiple times.

## Function Parameters & Arguments

- **Parameters** are placeholders for input values when defining a function.
- **Arguments** are the actual values passed when calling a function.

### **Example:**

```
function greetUser(userName, message) {  
    console.log(userName + ": " + message);  
}
```

```
greetUser("Max", "Hello!");  
greetUser("Manuel", "Hello, what's up?");
```

## Output:

Max: Hello!

Manuel: Hello, what's up?

- **Default Parameters:** If no value is provided, a default is used.

```
function greetUser(userName, message = "Hello!") {  
  console.log(userName + ": " + message);  
}
```

```
greetUser("Max"); // Output: Max: Hello!
```

## Returning Values

- The `return` keyword allows a function to output a value.
- Returned values can be stored in **variables** for later use.

## Example:

```
function createGreeting(userName, message) {  
  return "Hi, I am " + userName + ". " + message;  
}
```

```
const greeting1 = createGreeting("Max", "Hello!");  
const greeting2 = createGreeting("Manuel", "Hello,  
what's up?");
```

```
console.log(greeting1);  
console.log(greeting2);
```

## Output:

Hi, I am Max. Hello!

Hi, I am Manuel. Hello, what's up?

## **Best Practices**

- **Function names** should clearly describe their purpose.
- Use **CamelCase** notation for multi-word names (`createGreeting`).
- Functions should be reusable and follow **single responsibility principle** (one function, one purpose).

# JavaScript Functions & Arrow Functions

## Function Declarations

- Functions are **blocks of code** that are **not executed immediately** but are **invoked when needed**.
- Functions can be created using the **function keyword** or the **arrow function syntax**.

## Basic Function Syntax

```
function greet() {  
    console.log("Hello!");  
}  
greet(); // Invoking the function
```

- **Function Keyword:** Used to define a function.
- **Function Name:** Comes after the `function` keyword.
- **Parentheses ():** Can contain parameters.
- **Curly Braces {}:** Contain function logic.
- **Invocation:** Calling the function executes its code.

## Function Parameters & Arguments

- **Parameters:** Input values accepted by functions.
- **Arguments:** Values passed to parameters during invocation.

```
function greetUser(username, message) {  
    console.log(username + ": " + message);  
}  
greetUser("Max", "Hello!"); // Output: Max:  
Hello!
```

- **Multiple Parameters:** Separated by commas , .
- **Default Parameters:** If a value isn't provided, a default is used.

```
function greetUser(username, message = "Hello!")
{
  console.log(username + ": " + message);
}
greetUser("Max"); // Output: Max: Hello!
```

## Returning Values from Functions

- Functions can return values using the **return** keyword.

```
function createGreeting(username, message) {
  return "Hi, I am " + username + ". " + message;
}
const greeting = createGreeting("Max", "Nice to
meet you!");
console.log(greeting); // Output: Hi, I am Max.
Nice to meet you!
```

- **Returned values** can be stored in variables.
- **Function Naming:** Names should describe what the function does.

## Arrow Functions (=>)

- **Shorter syntax** for defining functions.
- **Commonly used for anonymous functions** (functions without names).

## Arrow Function Syntax

```
const greetUser = (username, message) => {
  return username + ": " + message;
};
console.log(greetUser("Max", "Hello!")); // Output:
Max: Hello!
```

- **Omit the function keyword.**
- **Arrow (=>)** replaces `function`.
- **Curly braces { }** contain the function body.

## Implicit Return in Arrow Functions

- If a function has a **single expression**, `{ }` and `return` can be omitted.

```
const greetUser = (username, message) => username  
+ ": " + message;  
console.log(greetUser("Max", "Hello!")); //  
Output: Max: Hello!
```

## Comparison: Function vs. Arrow Function

| Feature             | Function Declaration               | Arrow Function                                    |
|---------------------|------------------------------------|---------------------------------------------------|
| <b>Syntax</b>       | Uses <code>function</code> keyword | Uses <code>=&gt;</code>                           |
| <b>Return</b>       | Must use <code>return</code>       | Implicit return for single expressions            |
| <b>this Binding</b> | Has its own <code>this</code>      | Inherits <code>this</code> from surrounding scope |
| <b>Usage</b>        | Good for methods and constructors  | Great for callbacks and concise functions         |

## Key Takeaways

- **Functions** allow reusable, modular code.
- **Arrow functions** provide a **shorter syntax**, useful for anonymous functions.
- **Use `return`** when a function needs to return a value.
- **Function names** should be descriptive of their purpose.





## JavaScript Objects and Classes

### Objects

- **Objects** group multiple values together using **key-value pairs**.
- Keys should follow **naming rules** and be **descriptive**.

Example:

```
const user = {  
  name: "Max",  
  age: 34  
};  
  
console.log(user); // Output: { name: "Max", age:  
34 }  
console.log(user.name); // Output: "Max"
```

- **Dot notation (.)** is used to access object properties.

### Methods (Functions in Objects)

- Objects can store **functions**, called **methods**.

Example:

```
const user = {  
  name: "Max",  
  age: 34,  
  greet() {  
    console.log("Hello!");  
  }  
};  
  
user.greet(); // Output: "Hello!"
```

- **this keyword** refers to the object itself and can access its properties.

Example:

```
const user = {
  name: "Max",
  age: 34,
  greet() {
    console.log(`Hello! I am ${this.age} years
old.`);
  }
};

user.greet(); // Output: "Hello! I am 34 years old."
```

## **Classes (Blueprints for Objects)**

- The **class keyword** creates a blueprint for objects.
- Class names should **start with a capital letter**.

Example:

```
class User {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }

  greet() {
    console.log("Hi!");
  }
}
```

```
const user1 = new User("Manuel", 35);  
console.log(user1); // Output: User { name:  
"Manuel", age: 35 }  
user1.greet(); // Output: "Hi!"
```

- The **constructor function** initializes object properties.
- **new keyword** creates an instance of the class.
- The created object has access to class **methods**.

## JavaScript Arrays & Utility Methods

### Arrays Overview

- **Arrays** are a crucial **JavaScript value type**.
- **Technically, arrays are objects**, but they store values in an **ordered list**, unlike objects that use **key-value pairs**.
- **Array Syntax:** Defined using **square brackets ([])**.

```
let hobbies = ["Sports", "Cooking", "Reading"];
```

### Accessing Array Elements

- Elements are accessed using **index notation (arrayName[index])**.
- **Indexing starts at 0**.

```
console.log(hobbies[0]); // Output: Sports
```

### Nested Arrays

- **Arrays can store any data types**, including **other arrays, objects, numbers, and strings**.

```
let nestedArray = ["Sports", "Cooking",  
  ["Reading", "Traveling"]];  
console.log(nestedArray[0][1]); // Output:  
Cooking
```

---

## Common JavaScript Array Methods

### **1. push () – Add Elements**

- **Appends an item to the end of an array.**

```
hobbies.push("Working");
console.log(hobbies);
// Output: ["Sports", "Cooking", "Reading",
"Working"]
```

## 2. `findIndex()` – Find an Element's Index

- Returns the **index** of the **first occurrence** of an element that meets the condition.
- Uses an **arrow function** as a callback.

```
let index = hobbies.findIndex(item => item ===
"Sports");
console.log(index); // Output: 0
```

- **How it works:**

- `findIndex()` loops through the array.
- Passes each element (`item`) to the **arrow function**.
- Returns `true` when the condition (`item === "Sports"`) is met.
- Outputs the **index** of the matched item.

## 3. `map()` – Transform an Array

- **Creates a new array** by applying a function to each element.
- **Does not modify the original array.**

```
let editedHobbies = hobbies.map(item => item +
"!");
console.log(editedHobbies);
// Output: ["Sports!", "Cooking!", "Reading!",
"Working!"]
```

- **Mapping to Objects:**

- Converts array elements into objects with a **custom key-value pair**.

```
let hobbyObjects = hobbies.map(item => ({
  text: item }));
console.log(hobbyObjects);
/* Output:
[
  { text: "Sports" },
  { text: "Cooking" },
  { text: "Reading" },
  { text: "Working" }
]
*/
```

**Note:**

- **Curly braces { }** define an object in JavaScript.
- To return an object from an **arrow function**, **wrap it in parentheses ( { })** to prevent it from being treated as the function body.

---

## **Key Takeaways**

- **Arrays store ordered lists of values** and are a **special type of object**.
- **Access elements using indexes (`array[index]`)**.
- **Common methods include:**
  - **`push()`** → Adds an element.
  - **`findIndex()`** → Finds the index of an element.
  - **`map()`** → Transforms an array into a new array.
- **Arrow functions** make array methods **more concise** and **readable**.

# Destructuring in JavaScript

## Destructuring Arrays

- **Destructuring** allows extracting values from an array into separate **variables/constants**.
- Instead of accessing elements by **index**, destructuring provides a **cleaner** syntax.

### Example (Without Destructuring):

```
const userNameData = ["Max", "Schwarzmuller"];
const firstName = userNameData[0]; // Extract first
element
const lastName = userNameData[1];  // Extract second
element

console.log(firstName); // Output: Max
console.log(lastName);  // Output: Schwarzmuller
```

### Example (With Destructuring):

```
const userNameData = ["Max", "Schwarzmuller"];
const [firstName, lastName] = userNameData; //
Destructuring

console.log(firstName); // Output: Max
console.log(lastName);  // Output: Schwarzmuller
```

### ✓ Key Points:

- **Square brackets [ ]** on the left indicate array destructuring.
- **Order matters**, values are assigned based on their position.
- **Shorter & more readable** compared to accessing elements manually.



---

## Destructuring Objects

- Object **destructuring** extracts properties into variables/constants **by property name**.
- Unlike arrays, **order doesn't matter** in objects.

### Example (Without Destructuring):

```
const user = { name: "Max", age: 30 };  
const userName = user.name;  
const userAge = user.age;
```

```
console.log(userName); // Output: Max  
console.log(userAge);  // Output: 30
```

### Example (With Destructuring):

```
const user = { name: "Max", age: 30 };  
const { name, age } = user; // Destructuring
```

```
console.log(name); // Output: Max  
console.log(age);  // Output: 30
```

### Key Points:

- **Curly braces { }** indicate object destructuring.
- Variable names **must match** object property names.
- **Order does not matter**, unlike arrays.

---

## Using Aliases in Object Destructuring

- **Aliases** rename extracted properties using `:.`

### Example (With Aliases):

```
const user = { name: "Max", age: 30 };  
const { name: userName, age: userAge } = user; //  
Assigning alias
```

```
console.log(userName); // Output: Max  
console.log(userAge);  // Output: 30
```

### ✅ Key Points:

- **Property name stays the same**, but we assign a **new variable name**.
- Helps avoid **naming conflicts** when multiple objects have similar properties.

---

## Summary

| Feature           | Arrays ([ ])                             | Objects ( { })                              |
|-------------------|------------------------------------------|---------------------------------------------|
| Extraction Method | <b>By position</b>                       | <b>By property name</b>                     |
| Syntax            | <code>const [var1, var2] = array;</code> | <code>const { key1, key2 } = object;</code> |
| Order Matters?    | ✅ <b>Yes</b>                             | ❌ <b>No</b>                                 |
| Custom Names?     | ✅ <b>Yes</b>                             | ❌ <b>Must match property name</b>           |
| Aliases?          | ❌ <b>Not needed</b>                      | ✅ <code>key: aliasName</code>               |

---

## Why Use Destructuring?

- ✓ **Improves readability**
- ✓ **Reduces repetitive code**
- ✓ **More concise than manual extraction**

This feature is **widely used** in modern JavaScript, especially in **React, Node.js, and APIs**.

## Spread Operator in JavaScript

### Spread Operator for Arrays

- The **spread operator** (`...`) allows spreading the **elements of an array** into a new array or object.
- It is useful for **merging arrays** or **copying arrays** without nesting them.

#### Example (Merging Arrays):

```
const hobbies = ["sports", "cooking"];
const newHobbies = ["reading"];
const mergedHobbies = [...hobbies, ...newHobbies]; //
Using spread operator
```

```
console.log(mergedHobbies); // Output: ["sports",
"cooking", "reading"]
```

#### ✅ Key Points:

- **Three dots** `...` pull out elements from an array and add them as **separate values**.
- If spread is **not used**, nested arrays are created.

#### Example (Without Spread):

```
const mergedHobbiesWithoutSpread = [hobbies,
newHobbies]; // Nested arrays
console.log(mergedHobbiesWithoutSpread); // Output:
[["sports", "cooking"], ["reading"]]
```

---

## Spread Operator for Objects

- The **spread operator** is also available for **objects**, allowing you to merge or clone properties from one object into another.

### Example (Merging Objects):

```
const user = { name: "Max", age: 30 };  
const extendedUser = { ...user, isAdmin: true }; //  
Using spread operator  
  
console.log(extendedUser);  
// Output: { name: "Max", age: 30, isAdmin: true }
```

### ✓ Key Points:

- The **spread operator** pulls out **key-value pairs** from an object and adds them to a new object.
- This helps to **merge objects** or **extend** existing ones.

---

## Summary of Spread Operator Usage

| Type     | Arrays ([ ])                                          | Objects ( { })                                 |
|----------|-------------------------------------------------------|------------------------------------------------|
| Syntax   | <code>const newArray = [...array1, ...array2];</code> | <code>const newObject = {...object1};</code>   |
| Use Case | Merging or copying arrays                             | Merging or copying objects                     |
| Behavior | Spreads <b>elements</b> into the new array            | Spreads <b>key-value pairs</b> into new object |
| Result   | Flat array without nested arrays                      | New object with merged properties              |

---

## **Why Use the Spread Operator?**

- ✓ **Simplifies merging arrays/objects**
- ✓ **Avoids mutation** (creates copies instead of modifying original arrays/objects)
- ✓ **More concise** than manually copying elements or properties.

This operator is essential for **manipulating data** in modern JavaScript.

# Control Structures in JavaScript

## 1. If Statement

- The **if statement** executes code **only if** a condition is met.
- It can be extended with:
  - **else** → executes code if the condition is **not met**.
  - **else if** → checks **additional conditions** if the previous condition wasn't met.
- **Triple equals (===)** ensures **strict comparison** (checks both **value and type**).

### Example (If-Else If-Else Structure):

```
const password = prompt("Enter your password:"); //
Get user input

if (password === "hello") {
    console.log("Access granted: hello works");
} else if (password === "Hello") {
    console.log("Access granted: Hello works");
} else {
    console.log("Access not granted");
}
```

### ✓ Key Points:

- **prompt()** gets user input from the browser.
- **Conditions are evaluated in order**; the first matching condition executes.
- **Only one block executes**, and the rest are ignored.

---

## 2. For Loop

- Loops execute **the same code multiple times**.
- A **for loop** is often used to iterate over arrays.

### Example (Basic For Loop with an Array):

```
const hobbies = ["sports", "cooking"];

for (const hobby of hobbies) {
  console.log(hobby);
}
```

### ✓ Key Points:

- **for...of loop** creates a **new constant** (hobby) for each item in hobbies.
- **Executes once per array element.**
- **Output:**
  - sports
  - cooking

---

## Summary of If Statements and For Loops

| Control Structure   | Purpose                                  | Example Syntax                                                |
|---------------------|------------------------------------------|---------------------------------------------------------------|
| <b>If Statement</b> | Executes code only if a condition is met | <pre>if (x &gt; 5) {   console.log(x); }</pre>                |
| <b>Else If</b>      | Checks additional conditions             | <pre>else if (x === 5) {   console.log("x is 5"); }</pre>     |
| <b>Else</b>         | Executes if no conditions are met        | <pre>else { console.log("x is not 5"); }</pre>                |
| <b>For Loop</b>     | Iterates through an array                | <pre>for (const item of array) {   console.log(item); }</pre> |



These control structures **help manage the flow of execution** in JavaScript programs.

## Interacting with the DOM in JavaScript vs. React

### 1. Manipulating the DOM in JavaScript

- JavaScript can **select, read, and change** elements in the **DOM (Document Object Model)**.
- This allows **runtime modifications** to a webpage.

#### **Example (Selecting and Removing an Element):**

```
const element = document.querySelector("#myElement");  
// Select an element  
element.remove(); // Remove the element from the DOM
```

#### **Key Points:**

- `querySelector()` selects an element by **CSS selector**.
  - `.remove()` removes the selected element from the page.
- 

### 2. React vs. Manual DOM Manipulation

- **React manages the DOM** using a **declarative approach**.
- Instead of **directly modifying the DOM**, React **re-renders** components based on state changes.
- **React handles updates efficiently** using the **Virtual DOM**.

#### **Example (React Handling UI Changes):**

```
import { useState } from "react";

function App() {
  const [isVisible, setIsVisible] = useState(true);

  return (
    <div>
      {isVisible && <p>Hello, World!</p>}
      <button onClick={() =>
setIsVisible(false)}>Remove Text</button>
    </div>
  );
}

export default App;
```

### ✓ Key Points:

- **useState** manages visibility.
- **No manual DOM manipulation**; React **re-renders components** when state changes.

---

## Summary: JavaScript vs. React for DOM Manipulation

| Approach                       | How it Works                                | Example                                                      |
|--------------------------------|---------------------------------------------|--------------------------------------------------------------|
| <b>JavaScript (Imperative)</b> | Selects elements manually and modifies them | <code>document.querySelector("#id").remove();</code>         |
| <b>React (Declarative)</b>     | Uses <b>state</b> to update the UI          | <code>{isVisible &amp;&amp; &lt;p&gt;Hello&lt;/p&gt;}</code> |

|  |                 |  |
|--|-----------------|--|
|  | dynamical<br>ly |  |
|--|-----------------|--|

### **Key Takeaway:**

React eliminates the need for **manual DOM manipulation**, making UI updates more efficient and predictable.

# Passing Functions as Values in JavaScript

## 1. Functions as First-Class Citizens

- **Functions in JavaScript are first-class citizens**, meaning they can be:
    - **Stored in variables**
    - **Passed as arguments to other functions**
    - **Returned from other functions**
  - This allows **higher-order functions**, where one function **accepts another function as an argument**.
- 

## 2. Using `setTimeout()` with Function Arguments

- **`setTimeout()`** is a built-in JavaScript function that:
  - **Accepts a function as its first argument.**
  - **Executes the function after a specified delay (in milliseconds).**

### ✓ **Example: Passing a Function to `setTimeout()`**

```
function handleTimeout() {  
  console.log("Timed out!");  
}
```

```
// Passing function reference (without parentheses)  
setTimeout(handleTimeout, 2000); // Executes after 2  
seconds
```

### ✓ **Key Points:**

- **Do not use parentheses (`handleTimeout()`)** when passing the function.

- If parentheses are used, the function executes **immediately**, passing its **return value** instead of the function itself.
- 

### **3. Anonymous Functions vs. Named Functions**

- **Anonymous functions:** Functions **without a name**.
- **Named functions:** Functions with a **defined identifier**.

#### **Example: Using an Anonymous Function in `setTimeout()`**

```
setTimeout(function () {  
    console.log("Anonymous timeout");  
}, 2000);
```

#### **Example: Using an Arrow Function in `setTimeout()`**

```
setTimeout(() => console.log("Arrow function  
timeout"), 2000);
```

**Arrow functions are always anonymous**, but can be stored in variables to give them a name.

#### **Example: Storing an Arrow Function in a Variable**

```
const handleTimeout2 = () => console.log("Stored  
arrow function timeout");  
setTimeout(handleTimeout2, 2000);
```

---

### **4. Passing Functions to Custom Functions**

- Custom functions can also accept functions as arguments, similar to `setTimeout()`.

### ✓ Example: Passing a Function to a Custom Function

```
function greeter(greetFn) {  
    greetFn(); // Executes the function passed as an  
    argument  
}  
  
greeter(() => console.log("Hi"));
```

### ✓ Key Points:

- The **greeter** function accepts a function (`greetFn`) as a parameter.
- The function is **executed inside greeter()** using `greetFn()`.

---

## Summary: Function Passing in JavaScript

| Concept                                       | Explanation                                                 | Example                                                                     |
|-----------------------------------------------|-------------------------------------------------------------|-----------------------------------------------------------------------------|
| <b>Passing a function to another function</b> | Functions can be passed as <b>values</b>                    | <code>setTimeout(handleTimeout, 2000);</code>                               |
| <b>Using anonymous functions</b>              | Functions <b>without a name</b> , often used inline         | <code>setTimeout(() =&gt; console.log("Done"), 1000);</code>                |
| <b>Using named functions</b>                  | Functions <b>stored in a variable or defined separately</b> | <code>const myFn = () =&gt; console.log("Hello");<br/>greeter(myFn);</code> |
| <b>Custom function</b>                        | Any function can accept                                     | <code>function greeter(fn) {<br/>    fn();<br/>}</code>                     |

|                                           |                                    |  |
|-------------------------------------------|------------------------------------|--|
| <b>accepting<br/>another<br/>function</b> | another function<br>as an argument |  |
|-------------------------------------------|------------------------------------|--|

### **Key Takeaway:**

Passing functions as values enables **flexible, reusable, and modular** code. It is a core feature of JavaScript, not just limited to built-in functions like `setTimeout()`.



## Functions Inside Functions (Nested Functions)

- **Functions can be defined inside other functions.**
  - This is common in **React** and certain JavaScript patterns.
  - The inner function is **scoped** to the outer function and cannot be accessed outside.

### Example

```
function init() {  
  function greet() {  
    console.log("Hi");  
  }  
  greet(); // ✅ Can be called inside init  
}  
  
init(); // ✅ Calls init, which executes greet  
  
// greet(); ❌ Error: greet is not defined  
outside init
```

- **greet is only available inside init** (similar to how variables declared inside a function are not accessible outside).
- Calling `init()` executes `greet()` internally.

✅ **Practical Use:** While not commonly used in plain JavaScript, **nested functions** are **useful in React** for encapsulating logic within components.

## Reference vs. Primitive Values

- **Primitive values:** Include **strings, numbers, and Booleans**.
  - **Immutable** – cannot be modified once created.
  - **Reassignment creates a new value**; the old one is discarded.

### Example

```
let userMessage = "Hi";
userMessage = "Hello"; // Creates a new string;
                        "Hi" is discarded
```

- **Even string methods return new values instead of modifying the original string:**

```
let text = "Hello";
let newText = text.concat(" World");
console.log(text);    // "Hello" (unchanged)
console.log(newText); // "Hello World" (new
                      string)
```

---

## Reference Values (Objects & Arrays)

- **Objects and arrays are reference values.**
  - **Stored by reference (memory address), not by value.**
  - **Modifications affect the original object** instead of creating a new one.

### Example

```
const hobbies = ["Sports", "Cooking"];
hobbies.push("Working"); //  Modifies the
original array
```

```
console.log(hobbies); // ["Sports", "Cooking",  
"Working"]
```

- **Even though `hobbies` is a `const`, it can be modified.**
  - `const` **prevents reassignment** (e.g., `hobbies = []` ❌).
  - But **modifying values inside is allowed** because the memory **address** remains the same.

### Why?

- **Primitives store the value directly in the variable.**
- **Objects store a reference (memory address) instead of the actual value.**
- **Modifying an object changes the value at that memory address, not the reference itself.**

✅ **Key Concept: `const` prevents reassignment, not mutation.**